

AI 빅데이터



파이썬 & AI 프로그래밍 기초

초보자를 위한 입문 강의

강사: 김미선 | 2026년 6월 29일
파이썬의 기초부터 실습까지, 함께 시작해봅시다!

AI 빅데이터

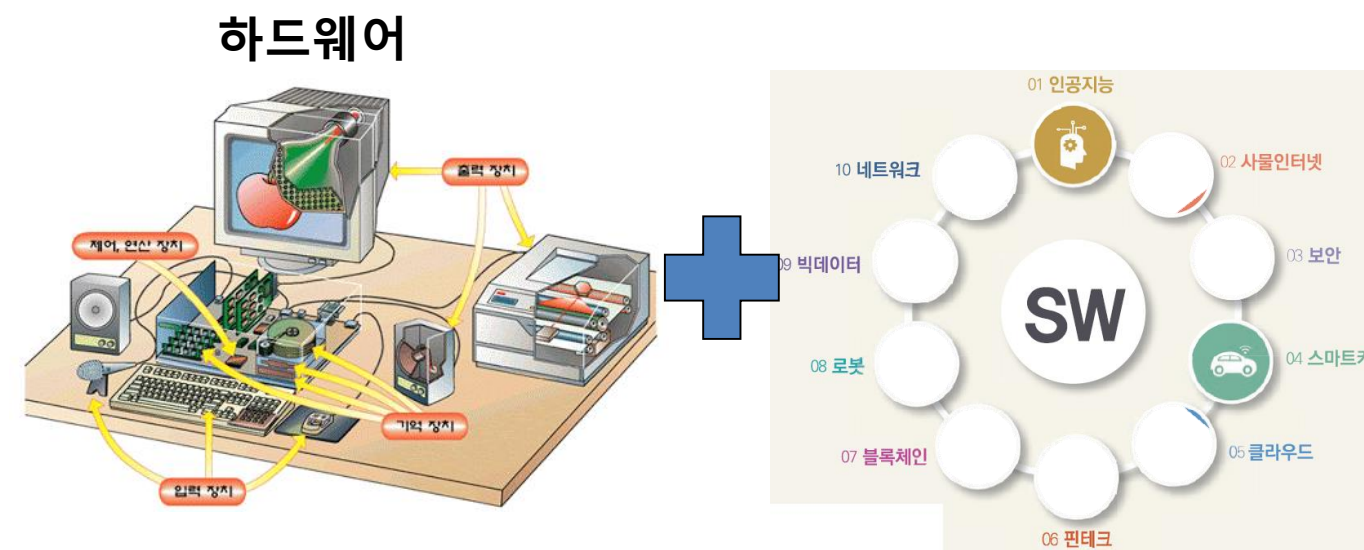


AI 코딩 개요

✓ 생성형 AI와 LLM 이해

AI (Artificial Intelligence)

- 컴퓨터로 하여금 “지능적 행동”을 하게하는 기술
- 컴퓨터를 더 스마트하게 ... 인지기능을 갖는 컴퓨터 만들기



- 컴퓨터에서 음성 및 작성된 언어를 보고 이해하고 번역하고 데이터를 분석하고 추천하는 기능을 포함하여 다양한 고급 기능을 수행할 수 있는 일련의 기술 (IBM)

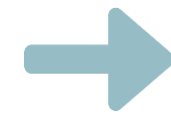
✓ 생성형 AI와 LLM 이해

AI (Artificial Intelligence)

AI는 어떻게 발전해 왔을까?

전통적인 AI
(규칙 기반 AI)

"이럴 땐 이렇게 해 "
사람이 모든 규칙을 직접
정해서 알려줌. 질문과 답
모두.



머신 러닝

사람이 입력한 데이터에서
스스로 규칙을 찾아냄.
고양이 사진 1000장
->특징을 찾아서
고양이를 구분하는 규칙을
만들어냄.



딥 러닝

사람의 뇌를 본 떠 만든
신경망 구조를 사용해서
더 복잡한 문제까지
해결할 수 있도록 학습함.

이 AI들은 분석이나 예측은 잘하지만 무엇가를 새롭게 만들어 내지 못함.

✓ 생성형 AI와 LLM 이해

생성형 AI

- 비정형 딥러닝 모델을 사용하여 사용자 입력을 기반으로 콘텐츠를 생성하는 인공지능 한 유형
- 이용자가 요구한 질문이나 과제를 해결하기 위해, 스스로 데이터를 찾아서 학습하여, 이를 토대로 능동적으로 데이터나 콘텐츠 등 결과물을 제시하는 한 단계 더 진화한 AI 기술
- 데이터로 콘텐츠나 사물을 학습하고, 이 데이터를 이용해 완전히 새로운 콘텐츠를 만드는 방법
- 생성 콘텐츠 : 텍스트, 이미지, 비디오, 오디오, 음악, 컴퓨터 코드
- 생성형 AI 종류
 - ✓ 대화형 AI 서비스 : ChatGPT, Gemini, Claude, Perplexity 등
 - ✓ 코드 생성·보조 AI 서비스 : Cusor, Copilot, Claude Code 등

✓ 생성형 AI와 LLM 이해

LLM(Large Language Model)

- 인터넷에 남긴 엄청난 양의 텍스트 데이터를 학습하여, *****다음에 올 가장 확률 높은 단어*****를 예측하는 초거대 AI 모델
- 수많은 문장과 자료를 학습해 단어와 문장 사이의 관계를 파악함으로써 답변을 생성
- 대표적 모델
 - ✓ ChatGPT(OpenAI), Claude(Anthropic), Gemini(Google) 등

✓ 생성형 AI와 LLM 이해

생성형 AI 챗봇 사이트

- 챗GPT(ChatGPT) (<https://chatgpt.com>)
- 제미나이(Gemini) (<https://gemini.google.com>)
- 클로드(Claude) (<https://claude.ai>)
- 그록 (<https://grok.com>)
- 퍼플렉시티(Perplexity) (<https://perplexity.ai>)
- 젠스파크(GenSpark) (<https://genspark.ai>)
- 라이너(Liner) (<https://getliner.com>)
- 펠로(Felo) (<https://felo.ai>)
- 마누스(manus) (<https://manus.im/>)

✓ 생성형 AI와 LLM 이해

분야별 AI 서비스 리스트

📝 글쓰기 및 전반적인 작업



📝 리서치 작업



💻 바이트 코딩



🎙️ 회의록 / 기록 작성



🖨️ 시각화 / PPT



🖨️ 웹/UI/UX 디자인



✓ 생성형 AI와 LLM 이해

분야별 AI 서비스 리스트

🖼️ 이미지 생성/편집


미드저니


ImageFX


Krea


Leonardo


ChatGPT


AI Studio


Photoshop


Magnific


Grok


Firefly


Genspark


Scenario


Playground


Freepik


Dreamina


Ideogram


Flux


Recraft

🎬 동영상 생성/편집


Kling


Sora


Veo3


Runway


Krea


Freepik


LTX Studio


Hailuo


Qwen


Genspark


Pika


Luma


Vidu


Higgsfields


Flora

🎬 동영상 편집


Capcut


Vrew


Veed


Invideo

👤 AI 아바타/립싱크


Heygen


Synthesis


Hedra

🔬 연구 및 리서치


Consensus


노트북LM


AlphaXiv


Storm

🧠 AI 모델


허깅페이스


Replicate


Fal AI


Tost AI

✓ 생성형 AI와 LLM 이해

분야별 AI 서비스 리스트

AI 만화 / 스토리보드



Anifusion



Novel AI



Stroy Tribe

자동화



Make



Dify



n8n



Zapier

음성 합성



ElevenLabs



타입캐스트

번역



DeepL



Genspark

음악생성



Suno



Udio



MusicFX

로컬AI



Pinokio



ComfyUI



Ollama



LmStudio

캐릭터 챗봇



크랙



캐릭터.AI

추론 AI



Groq



Cerebras



Together AI

AI 에이전트



Manus



Genspark



Flowwith

유틸리티/기타



Firecrawl

슬기로운 대학생활



뤼튼



Lilys AI

오픈소스

Super Vision Transformers OpenWebUI

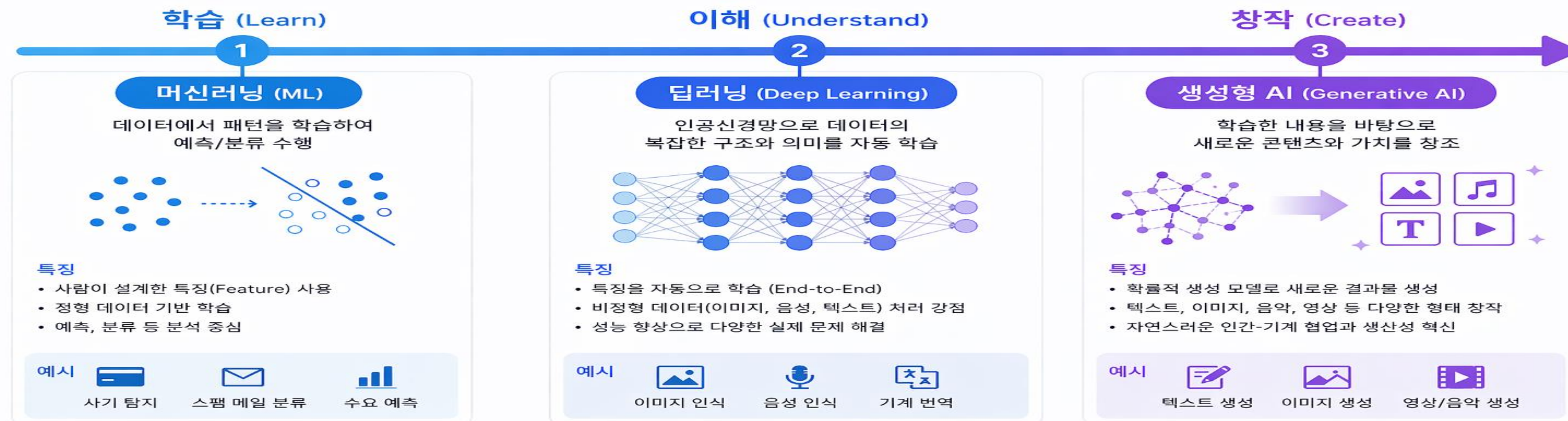
Hunyuan / Wan 2.1 Cline

✓ 생성형 AI와 LLM 이해

생성형 AI 활용 사례 : ChatGPT

AI 발전의 흐름: 학습에서 창작으로

데이터로 배우던 AI가 스스로 이해하고, 새로운 가치를 창조합니다.



데이터 학습 → 의미 이해 → 새로운 창작



AI는 단순한 도구를 넘어, 우리의 상상력을 확장하는 파트너로 진화하고 있습니다.

✓ 생성형 AI와 LLM 이해

생성형 AI 활용 사례 : Gemini

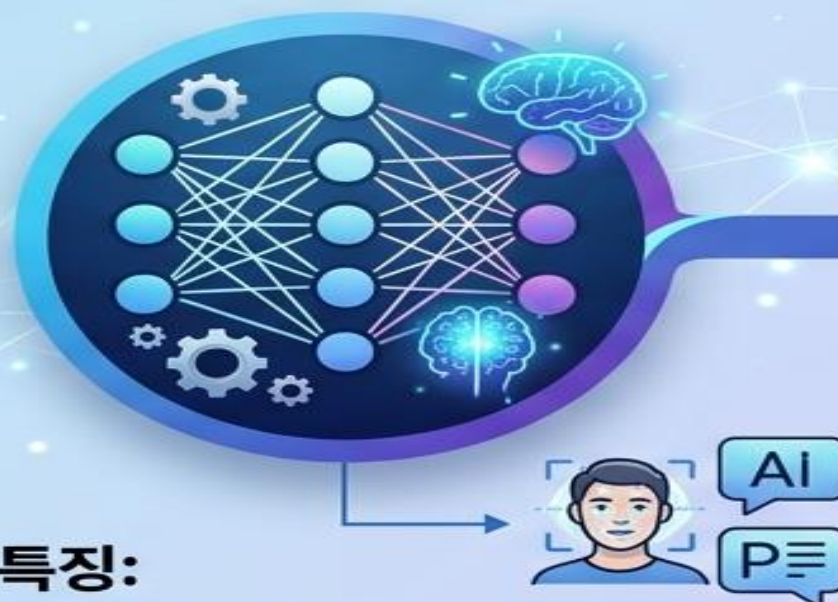
AI 발전의 흐름: 학습에서 창작으로 (EVOLUTION OF AI: FROM LEARNING TO CREATION)

1단계: 머신러닝 (MACHINE LEARNING) - 데이터 학습



- 특징:
- 데이터 기반 패턴 인식
 - 규칙 학습 및 분류
 - 성능 향상을 위한 데이터 필요

2단계: 딥러닝 (DEEP LEARNING) - 복잡한 패턴 인지



- 특징:
- 인공신경망 활용
 - 대규모 데이터와 컴퓨팅 파워
 - 이미지/음성 정밀 인식

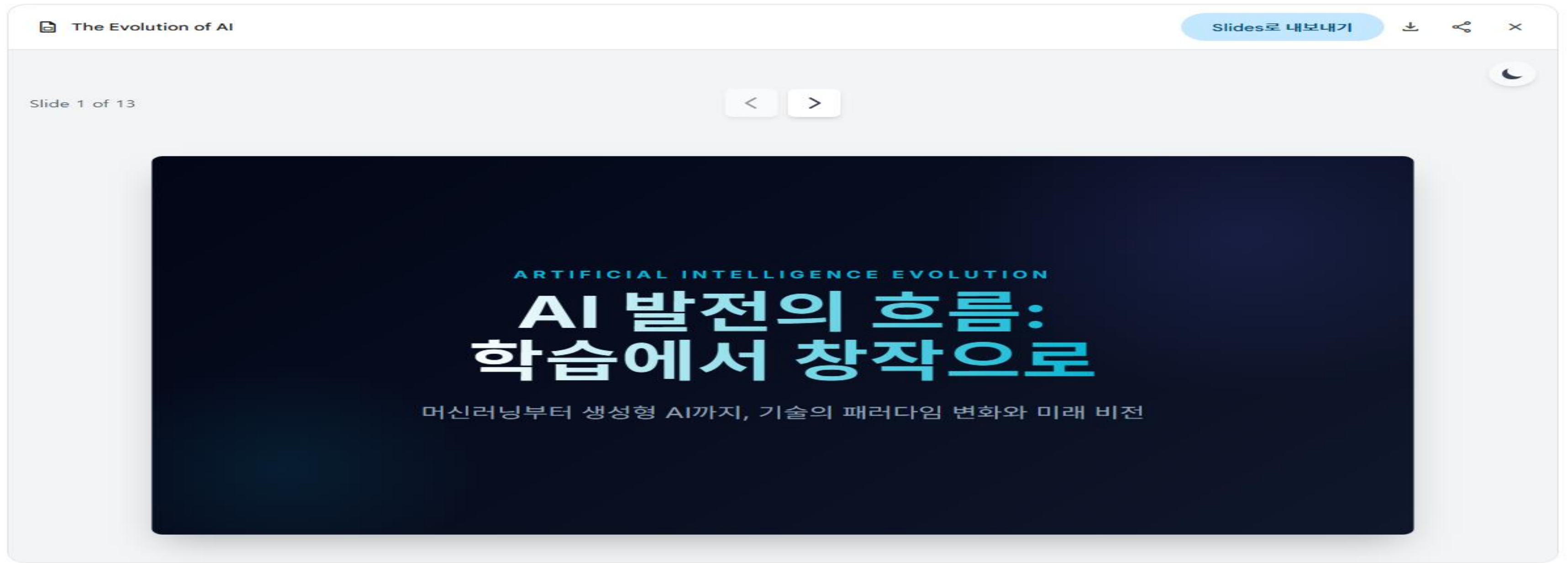
3단계: 생성형 AI (GENERATIVE AI) - 새로운 가치 창출



- 특징:
- 기존 데이터 기반 새로운 콘텐츠 생성
 - 문맥 이해 및 창의적 결과물
 - 트랜스포머 모델 등 혁신

✓ 생성형 AI와 LLM 이해

생성형 AI 활용 사례 : Gemini Canvas



✓ 생성형 AI와 LLM 이해

생성형 AI 활용 사례 : NotebookLM

인공지능(AI)의 진화: 태동에서 생성형 AI, 그리고 그 너머로

1950년대 앨런 튜링의 재안으로 시작된 AI는 머신러닝과 딥러닝을 거쳐 비약적으로 발전했습니다.
현재는 챗GPT를 필두로 한 생성형 AI 시대를 지나, 보안과 개인화에 최적화된 '몬디바이스 AI'로의 진화를 앞두고 있습니다.

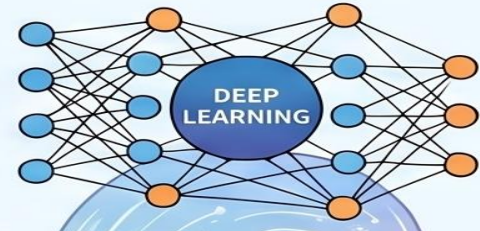
AI 발전의 역사적 이정표



1950s-1980s: AI 개념의 탄생과 초기 탐색

튜링 테스트와 다트머스 회의를 통해 '딥러닝' 분야가 처음으로 정립되었습니다.

**2010s:
딥러닝의 압도적 성능 증명**
알파고와 이미지 인식 확산과 말과 글의 승리로 본격적인 AI 시대가 열렸습니다.



2010s: 딥러닝의 압도적 성능 증명

알파고의 이미지 인식 확산과 말과 글의 승리로 본격적인 AI 시대가 열렸습니다.

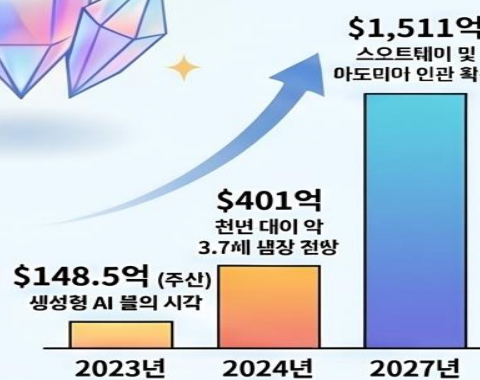


데이터와 GPU의 결합이 만든 도약

방대한 데이터와 GPU의 병렬 연산 기술이 딥러닝의 성숙을 견인했습니다.

생성형 AI 시대와 미래 전망

**2022년~현재:
생성형 AI와 창작의 영역**
챗GPT 출시 이후 텍스트를 넘어 이미지, 영상까지 생성하는 멀티모달 모델로 진화했습니다.



생성형 AI 시장의 급격한 성장 예측치

챗GPT 출시 이후 텍스트를 넘어 이미지, 영상까지 생성하는 멀티모달 모델로 진화했습니다.



차세대 핵심 기술 '몬디바이스 AI'

클라우드 임베딩 기반 자체 AI를 구동해 보안과 개인화 기능을 강화합니다.

보안 (Privacy)
개인화 (Personalization)



기술 발전에 따른 사회적 과제

인공지능을 통한 가짜 뉴스, 저작권 침해 및 일자리 대체에 대한 논의가 필요합니다.

✓ 생성형 AI와 LLM 이해

생성형 AI 활용

- 이번 수업을 통해 만들어 보고 싶은 프로그램 주제를 하나 찾아서 PRD(Product Requirements Document) 만들어보기 (주제 선정이 어렵다면, 이 부분도 AI에 요청해보기)

✓ AI 코딩(Vibe 코딩)

Vibe Coding

- 개발자가 문제를 간략하게 설명하면, AI가 필요한 코드를 알아서 생성해주는 AI 보조 프로그래밍 기법
- AI에게 내가 만들고 싶은 것의 느낌과 의도를 전달하여 코드를 구현하는 방식
- 조선공학도의 적용 예시
 - ✓ 전통적인 코딩 => `def calculate_buoyancy(rho, v): return rho * v` (직접 작성)
 - ✓ 바이브 코딩 => "해수 밀도를 고려해서 배수량을 계산하는 파이썬 함수를 짜줘. 입력값은 L, B, d야." (AI에게 지시)
- 프로그램을 하기 위해 우리에게 필요한 역량
 1. 무엇을 만들지 아는 능력 (전공 지식)
 2. 어떻게 설명할지 아는 능력 (프롬프트 엔지니어링)
 3. 결과가 맞는지 판단하는 능력 (검증)

✓ AI 코딩(Vibe 코딩)

Vibe Coding 성숙도 모델

- 1단계 : 코드 단위 보조
- 2단계 : 파일 단위 자동화
- 3단계 : 컴포넌트 단위 설계
- 4단계 : 프로젝트 단위 관리
- 5단계 : 서비스 단위 자동화 및 최적화

✓ AI 코딩(Vibe 코딩)

Vibe Coding

- 전략

- ✓ 명확한 요구사항 정의

- PRD(Product Requirements Document) 작성 을 통해 프로젝트의 목적, 주요 기능, 동작 방식등을 명확히 기술

- ✓ 단순화

- ✓ 작업의 작은 단위 분할

- PRD 를 기반으로 작업을 작은 단위로 분할

- ✓ 자동화된 검증과 관리

- ✓ 명확한 프롬프트와 규칙 제공

- ✓ 지속적인 개선과 반복

✓ AI 코딩(Vibe 코딩)

Vibe Coding

- Vibe Coder 역할
 - ✓ 명확한 방향 설정
 - ✓ 문제 발생시 적절한 대응 방안 제시
 - ✓ 세부 구현은 AI에게 맡기고 기획, 계획, 전략적 결정에 집중
- AI는 실행, 자료조사 -> 컨펌은 내가

✓ 프롬프트 엔지니어링

프롬프트란?

- 프롬프트(Prompt)
 - ✓ 생성형 AI에 지시하거나 질문할 때 입력하는 텍스트
- 프롬프트 엔지니어링(Prompt Engineering)
 - ✓ 대규모 언어 모델(LLM : Large Language Model) 에 지시를 내릴 때 더 나은 결과를 얻고자 프롬프트를 최적화하는 기술
 - ✓ 필요이유 : 특정 작업에 최적화된 프롬프트를 구성함으로써 대규모 언어 모델의 텍스트 생성 능력을 향상
 - ✓ 문제 해결 절차나 기법, 글쓰기 기법, 아이디어 발상법 등이 포함됨
 - ✓ 질문의 기술 (좋은 질문이 좋은 답변을 만듦)

✓ 프롬프트 엔지니어링

바이브 코딩을 위한 효과적인 프롬프트 작성 원칙

- 명확하고 구체적으로 지시하기
 - ✓ 코드 좀 개선해줘 -> 이 함수의 시간복잡도를 $O(n)$ 으로 줄이도록 리팩터링 하고, 결과를 순수 함수 형태로 작성해줘.
- 충분한 컨텍스트 제공하기
 - ✓ AI가 문제를 풀 때 필요한 배경지식과 의도를 프롬프트에 포함
- 프롬프트를 단계적으로 구조화하기
 - ✓ 복잡한 요구사항은 여러 단계로 나누거나 목록 형태로 지시
- 예시와 포맷 활용하기
 - ✓ 원하는 출력 형식이나 스타일이 있다면 샘플 예시를 프롬프트에 포함(퓨샷 프롬프트)
- AI에게 역할 부여하기
 - ✓ 특정 역할이나 관점을 심어주면 더 일관된 답변을 받을 수 있음 -> 조선 공학 전문 시니어 파이썬 프로그램 개발자
- AI의 사고를 유도하고 검증하기₩
 - ✓ 단계별로 생각을 전개하도록 질문을 던짐, 연쇄 사고(CoT) 유도

✓ 프롬프트 엔지니어링

프롬프트 작성 예시 비교

- 나쁜 프롬프트

- ✓ 선박 연료 소비량 및 운항 비용 예측 프로그램을 만들어줘

- 좋은 프롬프트

- ✓ 너는 국내 대형 조선소의 '친환경 스마트 선박 설계 부서'에서 근무하는 10년 차 수석 데이터 엔지니어이자, 파이썬 초보자를 교육하는 친절한 사수야.
- ✓ 우리는 선박의 운항 데이터를 바탕으로, 특정 항로를 운항할 때 소요되는 연료 소비량(Fuel Consumption)과 총 운항 비용(OPEX)을 예측하는 파이썬 프로그램을 개발하려고 해. 이 프로그램은 선주사에게 경제 운항 가이드를 제공하는 데 사용될 핵심 모듈이야.
- ✓ 조선공학 4학년 학부생(파이썬 초보자)이 구글 코랩(Google Colab)에서 바로 실행하고 이해할 수 있도록, 가상의 데이터를 다루는 데이터 전처리부터 머신러닝 예측, 비용 계산, 시각화까지 포함된 파이썬 스크립트를 작성해 줘.
- ✓ 코드는 반드시 다음 4단계 구조에 맞춰 나누어 작성하고, 단계별로 친절한 한글 주석을 달아줘.
 - 1단계 [데이터 준비 및 전처리]: NumPy/Pandas를 활용해 가상의 선박 운항 데이터셋(100개 행)을 생성해 줘. (포함될 변수: 엔진 RPM, 대지속력(Speed, Knot), 유의파고(Wave Height, m), 일일 연료소모량(Tons/day))
 - 2단계 [머신러닝 모델 학습]: Scikit-learn의 '랜덤 포레스트 회귀(Random Forest Regressor)' 모델을 활용해, RPM/속력/파고를 입력하면 일일 연료소모량을 예측하는 모델을 학습하고 RMSE 점수로 평가해 줘.
 - 3단계 [운항 비용 계산 함수]: 예측된 연료소모량과 항해 일수, 톤당 연료 가격(벙커유 가격)을 입력받아 총 연료 비용을 계산하는 파이썬 함수를 만들어 줘.
 - 4단계 [결과 시각화]: 실제 연료소모량과 모델의 예측값을 비교하는 산점도(Scatter Plot) 그래프를 Seaborn으로 그려줘. (한글 깨짐 방지용 폰트 설정 포함)

✓ 프롬프트 엔지니어링

프롬프트 작성 예시 비교

- 좋은 프롬프트
 - ✓ 최종 출력은 아래 양식을 엄격히 지켜서 답변해 줘.
 - `### 🚀 1. 프로그램 소스 코드`
 - ````python`
 - `# 전체 통합 코드를 여기에 작성 (Ready-to-Run 상태여야 함)`
 - ✓ 시뮬레이션을 위한 테스트 데이터 3개를 사용하여 테스트 후 출력 결과 예시 보여줘.
 - ✓ 코드를 최종적으로 출력하기 전에, 스스로 다음 내용을 검증(Self-Correction)하고 그 검증 과정을 한 문장으로 요약해 줘.
 - "파이썬 초보자가 코드를 복사해서 실행했을 때, 라이브러리 미설치나 문법 오류로 인해 에러가 날 가능성이 있는가?"
 - "조선공학적 상식선에서 RPM과 파고가 높아질 때 연료 소비량이 증가하는 경향이 가상 데이터에 잘 반영되었는가?"

✓ 프롬프트 엔지니어링

프롬프트 작성 실습 (복사해서 사용하세요)

- 너는 대한민국 대형 조선소(HD현대중공업, 삼성중공업, 한화오션 등)의 15년 차 '선체 설계 파트 수석 연구원'이야.
- 나는 이제 막 입사한 신입 사원이자 조선공학 전공자인데, 파이썬 코딩 역량은 기초적인 수준이야.
- 오늘 파트장님이 나에게 [여기에 원하는 조선공학 주제 입력, 예: 선체 중앙부의 단면계수(Section Modulus) 계산 / Simpson 규칙을 이용한 선면적 적분] 시스템을 파이썬으로 프로토타이핑하라고 지시하셨어.
- 선임 연구원인 네가 나를 위해 다음을 도와줘
- 1. 이 작업의 물리적 의미와 실무에서 왜 중요한지 짧게 리마인드해 줘.
- 2. 신입 사원인 내가 한 줄씩 읽고 이해할 수 있도록 '주석이 아주 상세히 달린' 파이썬 소스 코드를 작성해 줘.
- 3. 내가 이 코드를 실행할 때 입력해 볼 수 있는 '가상의 실무 데이터(테스트 케이스)' 예시도 함께 제공해 줘.

✓ 프로그램 개발 환경 설정

파이썬 프로그램 개발을 위한 준비

- 구글 코랩
- 파이썬 다운로드(아나콘다 다운로드하기)
- Visual Studio Code 다운로드
- 구글 계정 생성
- 깃허브 계정 생성 및 저장소 생성

AI 빅데이터



파이썬 프로그래밍 기초

프로그래밍 언어

- 사람의 의도를 컴퓨터가 이해할 수 있는 이진수(0,1)로 번역해 주는 도구
- 사람의 문제 해결 절차(알고리즘)를 컴퓨터가 실행 가능한 형태로 표현하는 규칙의 집합
- C, JAVA, Python
- 컴파일러 : 실행전에 소스코드를 기계어로 변환 (C, Java)
- 인터프리터 : 실행 시점에서 한줄씩 해석(Python)



문제를 “정확한 명령 + 데이터”로 바꾸는 과정이 프로그래밍입니다.



심플한 문법

간결한 코드

코드의 재사용

쉬운 유지보수



강력한 라이브러리

데이터 분석, AI, 웹 개발 등 다양한 기능을

지원하는 패키지 제공



거대한 커뮤니티

전 세계에서 가장 많이 사용되는 언어 중

하나

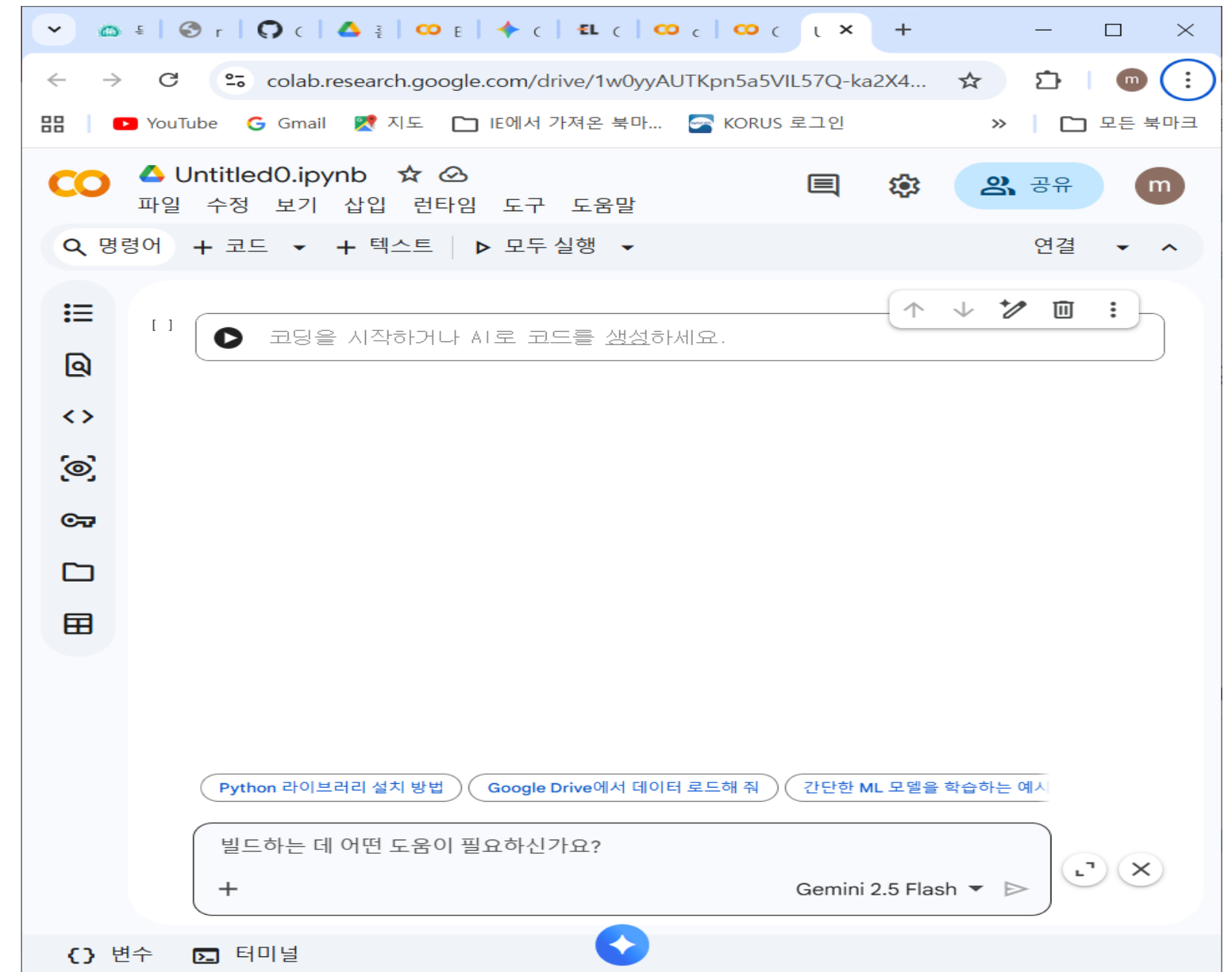
문제 발생 시 해결책을 찾기 매우 쉬움

아나콘다

- 파이썬 배포 플랫폼
- 파이썬 설치 파일 + 데이터분석 주요 패키지 + 주피터 노트북
- <https://www.anaconda.com/download/success> 에서 다운로드 가능
- 주피터 노트북을 사용하면 셀 단위 실행으로 코드를 조각조각 이해하기 좋음
- 결과가 바로 아래에 출력되어 학습 흐름이 자연스러움
- 데이터 시각화, 수식 등 교육 자료 만들기 용이
- AI 연동은 별도 설정 필요 (Continue.dev, Jupyter AI 등)

구글 코랩

- 웹브라우저에서 파이썬 코드를 실행할 수 있는 무료 클라우드 서비스(설치 없이 주피터노트북을 사용하여 파이썬 실행)
- 장점
 - ✓ 설치없이 바로 사용
 - ✓ 고성능 GPU를 무료로 사용
 - ✓ 코드 및 결과 공유
 - ✓ 다양한 라이브러리 사용
 - ✓ 작성한 코드가 구글 드라이브에 자동 저장
- 실행방법
 - ✓ 구글에서 코랩 검색하여 실행
 - ✓ <https://colab.google/> 사이트 연결



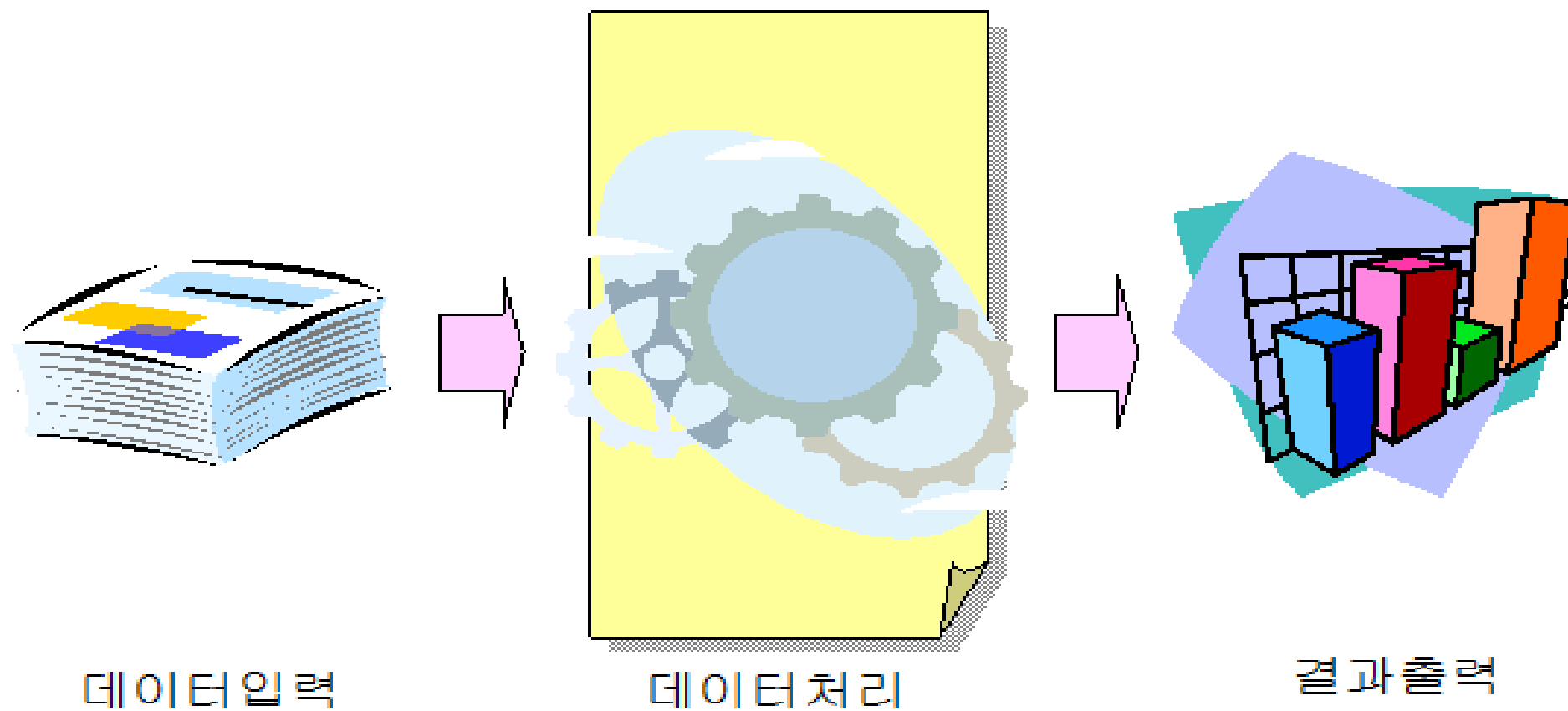
Visual Studio Code

- 가볍고 빠른 오픈소스 에디터
- <https://code.visualstudio.com/download>
- GitHub Copilot, Cursor 등 AI 도구와 가장 강력하게 통합됨
- 실제 개발 현장에서 가장 많이 쓰는 환경 → 실용적
- 초보자에게 UI가 복잡하고 설정할 게 많음
- Python 인터프리터 설정, 확장 설치 등 진입장벽 존재
- 설치 환경 차이로 인한 오류 대응 필요



프로그램의 구조

- 일반적인 프로그램은 외부로부터 데이터를 받아서(입력단계), 데이터를 처리한 후에(처리단계), 결과를 화면에 출력(출력단계)한다.

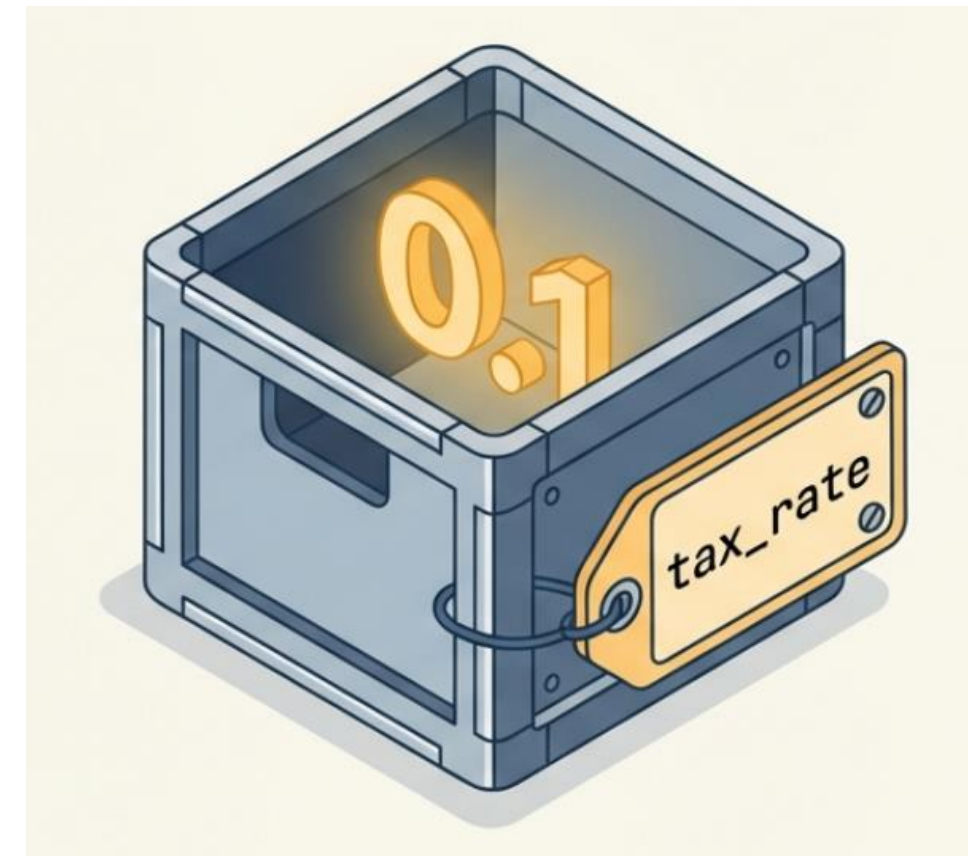


변수

- 데이터를 저장하는 바구니 역할
- 값을 부르는 '이름표'
- 변수는 값을 저장하는 공간인 동시에, 메모리에 있는 값에 붙인 "라벨(Label)"
- 값을 재사용하고 수정하기 쉽게 만듦
- 변수에 데이터 저장

▶ `tax_rate=0.1`
#세율이 바뀌면 이 숫자만 바꾸면 됨.

- = 는 "같다" 가 아니고 대입/할당의 의미
- 오른쪽의 데이터 또는 연산식의 결과를 왼쪽의 변수에 저장한다는 의미



변수 이름 규칙 & 스타일



Rule (규칙)

- ✓ 영문, 숫자, 언더스코어(_) 사용 가능.
- ⚠ **주의:** 숫자로 시작할 수 없음.
- ✗ **금지:** 예약어 (if, for, class 등) 사용 불가.



Style (스타일)



Best Practice

- ✓ 의미 있는 이름 사용
(a vs total_score).
- ✓ Snake Case 사용
(my_variable_name).

기본 자료형(Data Type)



int (정수)

10, -3
(소수점 없음)



float (실수)

3.14, 0.1
(소수점 있음)



str (문자열)

"Hello", "10"
(따옴표 필수)



bool (논리)

True, False
(참/거짓)



Pro Tip: `type(x)` 명령어로 **현재 값의 자료형**을 확인할 수 있습니다.

AI 활용

1. 변수 선언하고 출력하기

- 변수 정의: LOA, Breadth, Depth, Draft
- 프롬프트 : '선박 주요 제원 변수 정의 파이썬 코드 작성해줘'

2. 작성된 코드에 대해 수정사항 요청해보기

- 예) 변수에 값을 사용자로부터 입력받아 처리하기

화면에 문자열 출력

- print() 함수
 - 인자로 받은 문자열을 화면에 출력하는 함수
 - `print("Hello World")` => Hello World 출력
 - 문자열과 변수를 같이 출력할 때
 - `print(a,"+", b,"=", (a+b))` => 10 +20 =30 출력
 - `print("%d + %d = %d" %(a,b,a+b))` => 10 +20 =30 출력
- f-string 사용
 - Python 3.6부터 도입, 문자열 앞에 f를 붙여 사용
 - 중괄호 {} 안에 변수나 표현식을 직접 넣어 문자열을 포매팅
 - `print(f"이름: {name}, 나이: {age}")`
 - `print(f "{a} + {b} = {a+b} ")`
 - `print(f"가격: {price:,.0f}원")` # 천 단위 콤마, 소수점 없음
- str.format()
 - 문자열 내의 중괄호 {}를 플레이스홀더(placeholder)로 사용하고, format() 메서드의 인자로 전달된 값들로 이 플레이스홀더를 채움.
 - `print("이름: {}, 나이: {}".format(name,age))`

사용자로부터 입력받기

- input() 함수

- 사용자로부터 키보드를 통해 데이터를 입력받기 위한 함수

`name = input()` #사용자로부터 입력받은 문자열은 name 변수에 저장

`name = input("이름:")` #사용자에게 보여지는 메시지를 출력

- 입력받은 데이터는 문자열로 처리
- 문자열이 아닌 다른 자료형으로 사용하기 위해서는 형변환을 해줘야함
 - `score=int(input("점수를 입력하세요:"))` #문자열로 받아서 정수로 score 에 저장
 - `total=float(input("총점을 입력하세요: "))` #문자열로 받아서 실수로 total 에 저장

- 형변환 연산자

- `int()`, `float()`
- `bool()` # 숫자 0, 빈문자열 "" => False 그 외 True
- `str()`

연산자

- 산술연산자

연산자	의미	사용 예	설명
=	대입 연산자	<code>a = 3</code>	정수 3을 a에 대입
+	더하기	<code>a = 5 + 3</code>	5와 3을 더한 값을 a에 대입
-	빼기	<code>a = 5 - 3</code>	5에서 3을 뺀 값을 a에 대입
*	곱하기	<code>a = 5 * 3</code>	5와 3을 곱한 값을 a에 대입
/	나누기	<code>a = 5 / 3</code>	5를 3으로 나눈 값을 a에 대입
//	나누기(몫)	<code>a = 5 // 3</code>	5를 3으로 나눈 후 소수점을 버리고 값을 a에 대입
%	나머지값	<code>a = 5 % 3</code>	5를 3으로 나눈 후 나머지값을 a에 대입
**	제곱	<code>a = 5 ** 3</code>	5의 3제곱을 a에 대입

연산자

- 산술연산자와 대입 연산자
 - 대입 연산자 = 외에도 +=, -=, *=, /=, %=, //=, **= 사용
 - 예) 첫 번째 대입 연산자 a+=3은 a에 3을 더해서 다시 a에 넣으라는 의미로 a=a+3과 같음

표 4-2 대입 연산자의 종류

연산자	사용 예	설명
+=	a += 3	a = a + 3과 동일
-=	a -= 3	a = a - 3과 동일
*=	a *= 3	a = a * 3과 동일
/=	a /= 3	a = a / 3과 동일
//=	a //= 3	a = a // 3과 동일
%=	a %= 3	a = a % 3과 동일
**=	a **= 3	a = a ** 3과 동일

연산자

- 관계 연산자
 - 어떤 것이 크거나 작거나 같은지 비교하는 것(참은 True로, 거짓은 False로 표시)
 - 주로 조건문(if)이나 반복문(while)에서 사용, 단독으로는 거의 사용하지 않음

표 4-3 관계 연산자의 종류

연산자	의미	설명
==	같다.	두 값이 동일하면 참
!=	같지 않다.	두 값이 다르면 참
>	크다.	왼쪽이 크면 참
<	작다.	왼쪽이 작으면 참
>=	크거나 같다.	왼쪽이 크거나 같으면 참
<=	작거나 같다.	왼쪽이 작거나 같으면 참

연산자

- 논리 연산자
 - and(그리고), or(또는), not(부정) 세 가지 종류
 - 예) a라는 값이 100과 200 사이에 들어 있어야 한다는 조건 표현

표 4-4 논리 연산자의 종류

연산자	의미	설명	사용 예
and(논리곱)	~이고, 그리고	둘 다 참이어야 참	(a > 100) and (a < 200)
or(논리합)	~이거나, 또는	둘 중 하나만 참이어도 참	(a == 100) or (a == 200)
not(논리부정)	~아니다, 부정	참이면 거짓, 거짓이면 참	not(a < 100)

연산자

- 연산자 우선 순위
 - ✓ 여러 개의 연산자가 있을 경우 정해진 순서

표 4-6 연산자 우선순위

우선순위	연산자	의미
1	() [] {}	괄호, 리스트, 딕셔너리, 세트 등
2	**	지수
3	+ - ~	단항 연산자
4	* / % //	산술 연산자
5	+ -	
6	<< >>	비트 시프트 연산자
7	&	비트 논리곱
8	^	비트 배타적 논리합

9		비트 논리합
10	<> <= >=	관계 연산자
11	== !=	동등 연산자
12	= %= /= //= -= += *= **=	대입 연산자
13	not	논리 연산자
14	and	
15	or	
16	if ~ else	비교식

AI 활용

1. 사용자로부터 정수를 2개 입력 받아 사칙 연산을 수행하기

- 프롬프트
 - ✓ 사용자에게 정수를 2개 입력받아서 사칙연산을 수행하고 결과를 출력해줘.
 - ✓ 제약조건 : 산술연산자외에 사용하지 않기
- 본인이 작성 가능한 수준이면 직접 작성해보기
- 본인이 작성한 코드에 대해서 리팩토링 프롬프트 적용해보기

AI 활용

2. 합계/평균 계산기

- 문제: 정수 2개를 입력받아 합계(sum)와 평균(avg)을 출력
- 조건: 평균은 실수로 출력(예: 4.0)
- LLM 활용: "입력/출력 예시 + 제약조건" 포함해 코드 생성 요청
- 완료 기준: 테스트 3개를 직접 실행해 통과시키기

AI 활용

3. 문자열 포매팅 영수증

- 문제: 상품명, 가격, 수량을 입력받아 영수증 형태로 출력
- 필수: f-string 사용
- 예: 사과 1000원 × 3개 → 총액 3000원
- LLM 활용: “출력 모양”을 먼저 보여주고 그대로 나오게 요청

AI 활용

4. 단위 변환기(리팩터링)

- 문제: cm를 입력받아 inch로 변환($1 \text{ inch} = 2.54 \text{ cm}$)
- 추가: 섭씨→화씨 변환도 선택 문제로 추가해보기
- LLM 활용: 1) 코드 초안 2) 변수명 개선 3) 주석/함수화 아이디어
- 완료 기준: 입력/출력 안내 문구까지 자연스럽게

AI 활용

5. "거스름돈 계산기" 만들기(몫/나머지 활용)

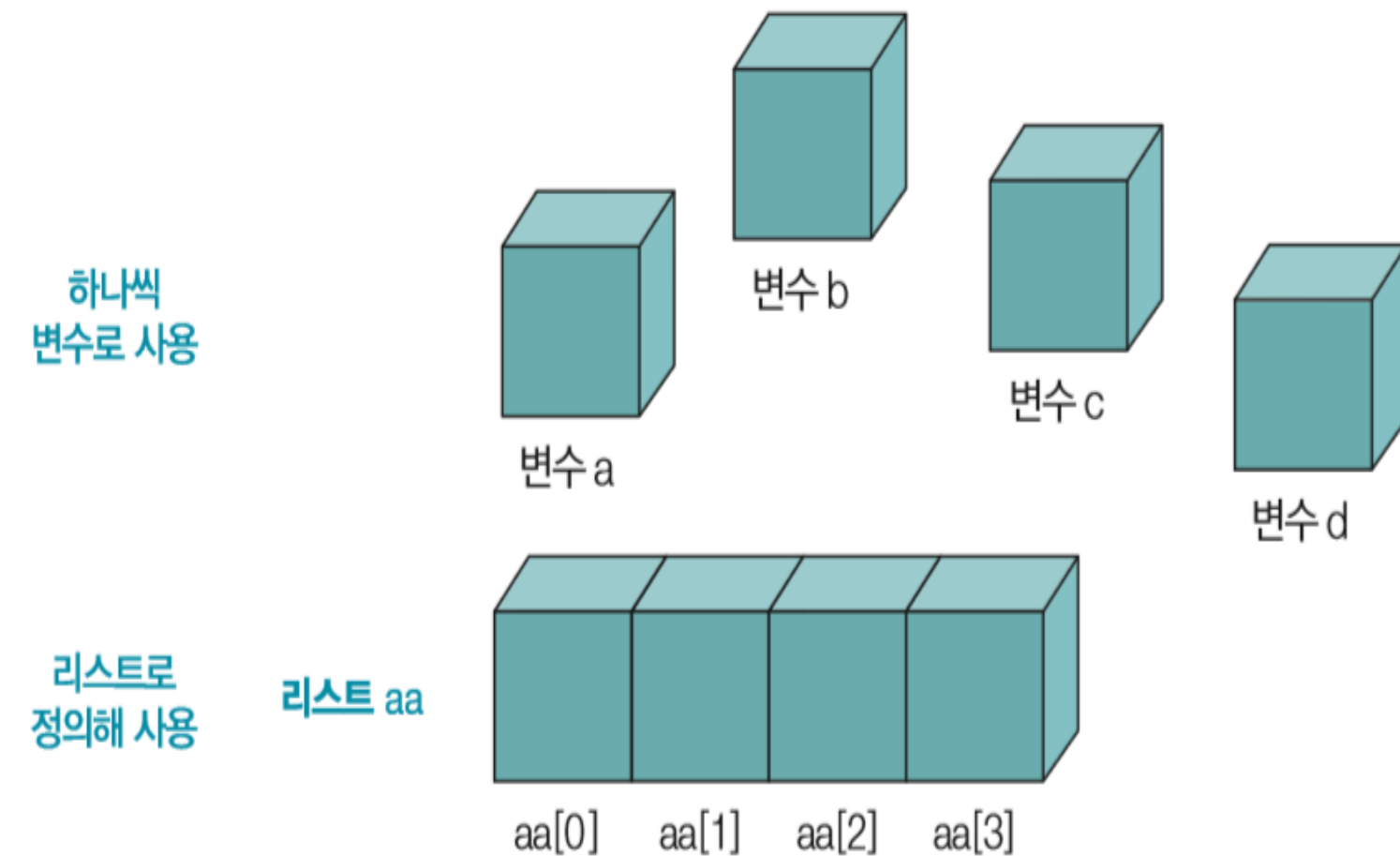
- 문제: 지불금액, 물품 금액을 입력 받아, 거스름돈을 출력
- 출력형식 : 5,000원권 : 0장 1,000원권 : 0장 500원 500원 동전 : 0개 100원 동전 : 3개 (각 권종당 줄바꿈으로 처리)
- 제약조건 : 지불금액은 만원단위로 입력
- LLM 활용: 1) 주석 및 테스트 데이터 3개 예시 제공
- 완료 기준: 입력/출력 안내 문구까지 자연스럽게

컨테이너 자료형

- 자료구조
 - ✓ 데이터를 저장하는 구조
- 파이썬에서는 여러 데이터를 효율적으로 관리할 수 있는 컨테이너형 자료구조 제공
 - ✓ list(리스트)
 - ✓ Tuple(튜플)
 - ✓ dictionary(사전)
 - ✓ set(집합)

리스트

- 데이터의 목록을 다루는 자료형
- 여러 값을 "순서대로" 저장하는 자료구조
- 원소(Element)는 서로 다른 자료형도 가능
- 리스트 정의시 [] 사용
- 각 요소 구별을 위해, 사용
- 각 요소 접근을 위해서는 첨자(인덱스) 사용
 - ✓ 인덱스는 0부터 시작
 - ✓ $a=[1,2,3,4] \Rightarrow a[0]$ 은 1
 - ✓ 음수 인덱스: 뒤에서부터(-1 마지막)
 - ✓ $a[-1]$ 은 4
- 리스트는 "수정 가능(mutable)"
- 인덱스 범위를 벗어나면 IndexError
- "여러 개 입력/여러 개 결과"는 리스트로 해결



리스트의 개념 (출처:파이썬 for Beginner)

리스트

- 슬라이싱
 - ✓ `a[1:3]` #1번 인덱스부터 3번 인덱스의 앞까지 (1-2), [2, 3]
 - ✓ `a[:3]` #0번 인덱스부터 3번 인덱스의 앞까지 (0-2), [1,2,3]
 - ✓ `a[::2]` #step 설정 (처음부터 끝까지 2칸씩 건너뛰면서), [1.3]
 - ✓ 슬라이싱 결과는 "새 리스트"
- 추가 /확장
 - ✓ `append(x)`: 맨 뒤에 1개 추가
 - ✓ `extend(iterable)`: 여러 개를 한 번에 추가
 - ✓ `insert(i, x)`: i 위치에 삽입
 - ✓ 주의: `append([1,2])`는 "리스트 1개"가 들어감

리스트

- 정렬

- ✓ `sort()`: 리스트 자체를 정렬(원본 변경)
- ✓ `sorted()`: 정렬된 "새 리스트" 반환(원본 유지)
- ✓ `reverse=True`로 내림차순
- ✓ `key=`로 정렬 기준 지정 가능(다음 주차에서 확장)

- 리스트의 장점

- ✓ 반복문과 함께 전체 값에 접근 가능
- ✓ `for (s in score):`

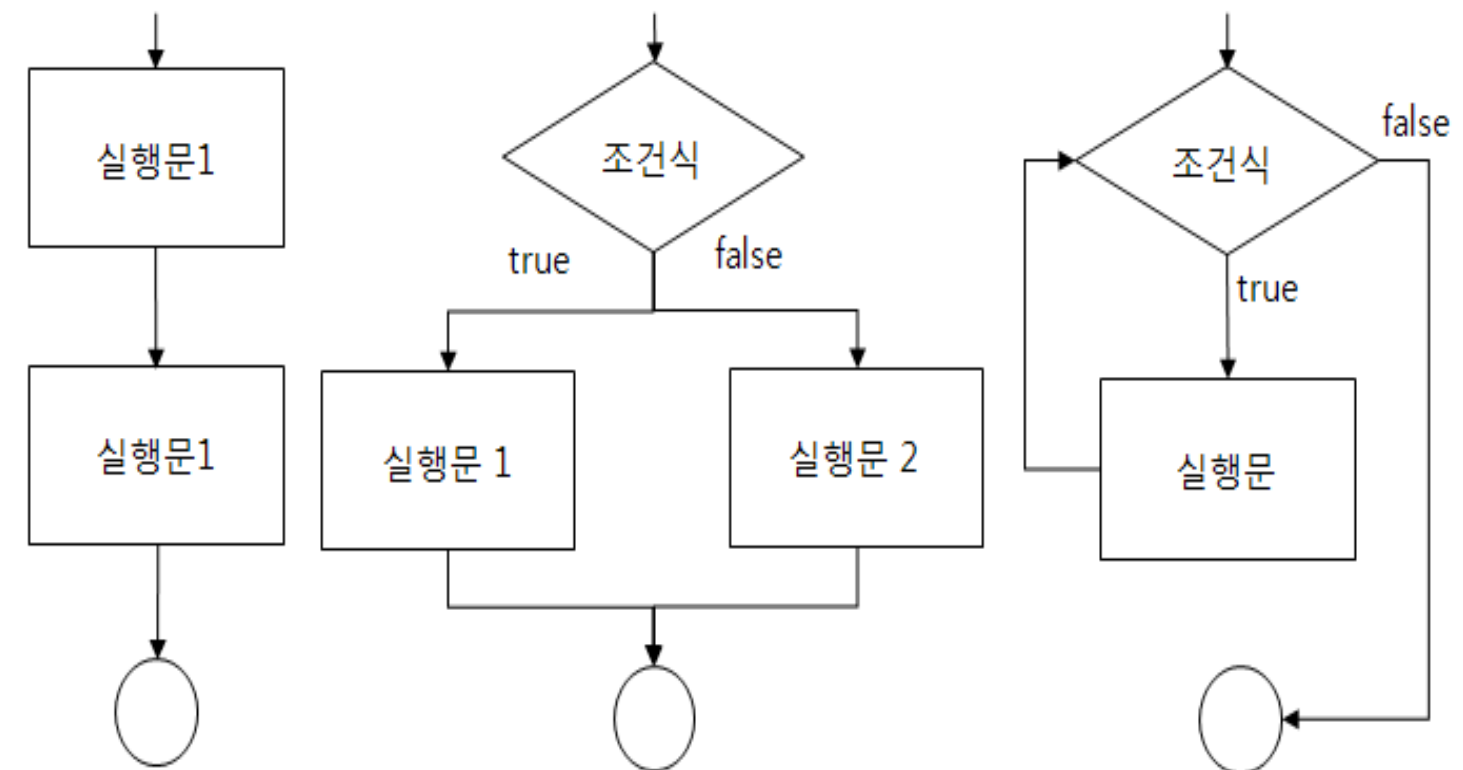
`sum+=s`

- 리스트의 복사

- ✓ `list1=list2` #list1과 list2가 같은 곳을 가리킴
- ✓ `list2=list1.copy()` # list1을 복사해서 list2에 저장

제어문

- 일반적 프로그램의 흐름
 - 위에서 아래로, 순차적인 순서의 흐름
- 제어구조
 - 다양한 문제의 해결을 위해 문장들이 실행되는 순서에 영향을 주는 문장이 필요
 - 순차구조 : 순차적인 순서의 흐름
 - 조건구조 : 조건에 따라 실행이 달라지는 구조
 - 반복구조 : 하나의 문장이 반복되는 구조



조건문

- if 문

- 조건식의 결과가 참(true)인지 거짓(false)인지에 따라 서로 다른 일들을 수행할 경우에 사용
- 단순 if 문이나 중첩 if 문의 형태로 사용
- 단순 if 문의 형식

if 조건식: 실행문	if a>0 : print(a , "는 양의 정수")
if 조건식: 실행문1 else: 실행문2	if a>0 : print(a , "는 양의 정수") else: print(a , "는 음의 정수")

- 파이썬은 코드 블록을 구분하기 위하여 {} 대신 들여쓰기를 사용한다. 제어문을 포함하여 코드 블록이 필요한 부분에서는 반드시 들여쓰기를 해야한다.

조건문

- 중첩 if 문
 - 여러 조건이 겹치는 경우 if 문의 if-block안에 또 다른 if 문을 넣고, 또 넣고 하는 식으로 작성할 수 있음

```
if 조건식1:
    실행문1
elif 조건식2:
    실행문2
else:
    실행문3
```

```
if 조건식1:
    실행문1
else:
    if 조건식2:
        실행문2
    else:
        실행문3
```

AI 활용

1. 성적 등급 계산기 (A/B/C/D/F + 범위 체크)

- 문제: 점수를 입력받아 학점 계산
- 입력: 0~100 점수(score)
- 출력: A(100-90), B(89-80), C(79-70), D(69-60), F(그 외)
- 추가: 범위 밖 입력이면 "입력 오류" 출력

반복문

- 프로그램의 일정한 부분(또는 실행문)에 대해 주어진 조건을 만족하는 동안 반복하는 문장
- 반복문이 필요한 이유
 - 일정부분을 반복하기 위해서는 순차구조로도 표현 가능하지만, 비효율적

```
while0.py - C:\Users\secu2019_15\Documents\while0.py (3.8.2)
File Edit Format Run Options Window Help
meter=0*1609
print("0 마일은 ",meter,"입니다.")
meter=1*1609
print("1 마일은 ",meter,"입니다.")
meter=2*1609
print("2 마일은 ",meter,"입니다.")
```

```
while00.py - C:/Users/secu2019_15/Documents/while00.py (3.8.2)
File Edit Format Run Options Window Help
mile=0
while mile<3:
    meter=mile*1609
    print(mile," 마일은 ",meter,"입니다.")
    mile+=1
|
```

for문

- for..in
 - 객체의 열거형(Sequence)을 따라서 반복하여 실행할 때 사용되는 반복문
 - for 변수명 in 반복자:
실행문
 - for i in range(1, 5):
- 열거형
 - 여러 항목이 나열된 어떤 목록을 의미
- range () 함수
 - range(x) : 0부터 x 미만까지 1씩 증가값 반환
 - range (x, y) : x부터 y미만까지 1씩 증가값 반환
 - range (x, y, z) : x부터 y미만까지 z씩 증가값 반환

AI 활용

1. 점수 분석기

- 점수 여러 개를 입력받아 리스트로 저장
- 출력: 평균/최대/최소/합계
- 도전: 90점 이상 개수 카운트
- LLM 활용: “테스트 입력”을 함께 만들기

while 문

- 특정 조건이 참일 경우 계속해서 블록의 명령문들을 반복하여 실행할 수 있도록 함
- 만약 조건이 거짓일 경우 while 문은 종료되고 다음 코드가 실행됨
- 반복의 횟수가 명확하지 않은 경우에 while 문 사용
- 사용자의 입력을 받아서 종료하는 경우, while True : 사용하여 무한루프
- while 조건식 :
실행문

흐름제어

- break
 - 루프(while, for) 문을 강제로 빠져나올 때, 즉 아직 루프 조건이 `false`가 되지 않았거나 열거형의 끝까지 루프가 도달하지 않았을 경우에 루프 문의 실행을 강제로 정지시키고 싶을 때 사용
- continue
 - 현재 실행중인 루프 블록의 나머지 명령문들을 실행하지 않고 곧바로 다음 루프로 넘어가도록 함

AI 활용

1. ToDo 리스트 만들기

- 리스트에 할 일을 저장
- add: 입력받아 append
- list: 현재 목록 출력(번호 포함)
- done: 번호를 입력받아 pop/remove로 삭제

AI 활용

2. 숫자 맞추기 게임

- 정답 숫자를 맞힐 때까지 반복 입력
- 힌트: 작으면 UP, 크면 DOWN
- 추가: 시도 횟수 카운트
- 무한 루프 방지: 시도 횟수 제한(max_tries) 같은 안전장치를 두어도 좋습니다.

01. 선박 제원 출력 및 배수량 계산 [산술연산자]

문제: 사용자의 입력을 받아 선박의 이름, 길이(L), 폭(B), 흘수(T)를 변수에 저장하고, 단순화된 공식을 이용해 배수량(Δ)을 계산하여 출력하는 프로그램을 작성하세요.

•공식: $\text{Displacement} = L * B * T * C_b * 1.025$ (단, 방형비 C_b 는 0.8로 가정)

02. 선체 구조 부재 무게 등급 판별 [제어문]

문제: 선박에 사용되는 강판의 무게를 입력받아, 무게에 따라 크레인 운송 등급(A, B, C)을 판별하는 프로그램을 작성하세요.

조건: 50톤 이상은 'A등급', 20톤 이상 50톤 미만은 'B등급', 20톤 미만은 'C등급'으로 분류합니다.

03. 강재 리스트 반복 출력 및 합계 [반복문]

문제: 리스트에 저장된 여러 개의 강재 무게 데이터를 반복문을 통해 하나씩 출력하고, 전체 강재의 총 무게를 계산하는 프로그램을 작성하세요.

왜 함수가 필요한가?

```
1 # 함수 없이: 같은 로직이 여러 번 반복
2 score = int(input('점수 입력: '))
3 if score >= 60:
4     print('PASS')
5 else:
6     print('FAIL')
7
8 score2 = int(input('점수 입력: '))
9 if score2 >= 60:
10    print('PASS')
11 else:
12    print('FAIL')
```

함수의 이점

- 중복 제거(DRY): 같은 코드를 한 곳에서만 관리
- 가독성 향상: '무엇을 하는지' 이름으로 설명 가능
- 재사용/확장: 다른 곳에서도 호출해서 사용
- 테스트 용이: 함수 단위로 검증(assert/테스트케이스)
- 협업에 유리: 역할(입력/처리/출력) 분리

함수 기본 문법(def)

```
1 def greet(name):  
2     """이름을 받아 인사 메시지를 반환"""  
3     msg = f"안녕하세요, {name}님!"  
4     return msg  
5  
6 result = greet('민수')  
7 print(result)
```

핵심 포인트

- 함수는 '작은 프로그램'(입력→처리→출력)이다.
- return은 '값'을 돌려준다. (print와 역할이 다름)
- 함수 이름은 동사로: greet, calculate, convert ...
- Docstring(""" """)은 사용법/의도를 기록한다.

매개변수·인자·반환값

```
1 def add(a, b): # a,b는 매개변수(parameter)
2     return a + b
3
4 x = add(2, 3) # 2,3은 인자(argument)
5 print(x)      # 5
6
7 # return이 없으면 None이 반환됨
```

정리

- 정의할 때: 매개변수(parameter) / 호출할 때: 인자(argument)
- return은 값을 '돌려주고' 함수는 종료된다.
- print는 화면 출력(부수효과)이고 값 반환과는 별개다.
- 초보 실수: return 대신 print만 해서 결과를 못 쓰는 경우

기본값·키워드 인자·가변 인자

```
1 def greet(name, msg='안녕'):
2     return f"{msg}, {name}!"
3
4 print(greet('철수'))
5 print(greet('영희', msg='반가워'))
6
7 def total(*nums):
8     return sum(nums)
9 print(total(1,2,3,4))
```

핵심 포인트

- 기본값(default)은 인자를 생략했을 때 사용된다.
- 키워드 인자로 '의도'를 드러낼 수 있다: msg='반가워'
- *args는 '여러 개의 위치 인자'를 튜플로 받는다(맞보기).
- **kwargs는 '여러 개의 키워드 인자'를 딕셔너리로 받는다(맞보기).

변수 범위(Scope)와 주의점

```
1 x = 10 # 전역 변수
2
3 def f():
4     x = 3 # 지역 변수(함수 안)
5     return x
6
7 print(f()) # 3
8 print(x)   # 10
9
10 # 전역 변경은 가급적 피하기
```

기억할 것

- 함수 안에서 만든 변수는 기본적으로 '지역'이다.
- 이름이 같아도 지역/전역은 서로 다른 변수일 수 있다.
- 전역을 변경해야 한다면 설계를 다시 생각해보자(부작용).
- LLM에게 '전역 변수 없이' 구현하라고 제약을 주면 안전하다.

함수 품질 체크

- 이름: 무엇을 하는지 한 눈에 보이게 (예: `calc_avg`, `load_csv`)
- 입력/출력: 함수가 받는 것과 돌려주는 것을 명확히
- 길이: 20~30줄 이상 길어지면 분해를 고려
- Docstring: 사용법/예시/예외 상황을 기록
- 타입 힌트(선택): 의도를 표현해 디버깅을 돕는다

Tip: '한 함수는 한 가지 일' 원칙을 연습해봅시다.

튜플

- 여러 개의 객체를 모아 담는데 사용
- 수정이 불가능, 읽기 전용 => 튜플 수정하고 싶은 경우, 리스트를 사용하여 튜플을 리스트로 변경 후, 다시 튜플로 변경
- ()를 사용하여 생성하거나, () 없이 생성가능
- 요소 구별은 , 사용
- 사용 예 : A=(1,2,3,4) A=1,2,3,4

세트

- 중복 요소를 허용하지 않으며 순서가 없는 데이터 묶음
- 순서에 상관없이 객체를 묶음 자체를 필요로 할 때 주로 사용
- 집합끼리는 멤버십 테스트를 통해 한 집합이 다른 집합의 부분집합인지 확인할 수 있으며, 두 집합의 교집합 등을 알아낼 수도 있음

딕셔너리

- 리스트와 비슷
- 리스트처럼 첨자를 사용하지만 리스트는 첨자를 숫자만 사용하는 반면, 사전은 문자열과 숫자를 비롯해서 변경이 불가능한 형식이면 어떤 자료형이든 사용
- 키와 값 한쌍으로 구성됨.
- 첨자로 사용되는 것이 키, 키는 유일한 값이어야 함
- 탐색 속도가 빠르고, 사용하기에도 편리
- 딕셔너리를 만들 때는 중괄호 { }을 이용
- 특정 위치에 새로운 키-값을 입력하거나 딕셔너리 안에 있는 요소를 참조할 때는 리스트와 튜플에서처럼 대괄호 [와]를 이용
- `d = { 'key1' : value1, 'key2' : value2 }`
- `d['key1']`처럼 접근 또는 `d.get('key1')`

딕셔너리

- 주로 사용하는 메소드
 - ✓ `get(key, default)`: 키가 없을 때 기본값
 - ✓ `items()`: (key,value) 쌍 반환
 - ✓ `keys()` : 딕셔너리에서 키만 반환
 - ✓ `values()` : 딕셔너리에서 값만 반환
- 자주 사용하는 기능
 - ✓ 키 존재 확인: `if key in d:`
 - ✓ 삭제: `pop(key)` 또는 `del d[key]`
 - ✓ 여러 값 추가/병합: `update(other_dict)`
 - ✓ 실무 팁: "키가 없을 가능성"부터 먼저 고려

AI 활용

1. 연락처 관리 (dict 사용)

- 이름을 키로, 번호를 값으로 하는 딕셔너리를 정의하여, 이를 이용한 연락처 관리 프로그램 구현
- 기능
 - ✓ add(이름, 번호)
 - ✓ find(이름)
 - ✓ list(전체 출력)
- 정책: 같은 이름이 있으면 '수정'하도록 구현

AI 활용

2. 로또 번호 검사 (set 활용)

- 사용자 입력 6개 (중복 없이)
- 당첨 번호와 교집합 개수 계산

표준 라이브러리 맛보기

```
1 import math
2 import random
3 from datetime import datetime
4
5 print(math.sqrt(16))          # 4.0
6 print(random.randint(1, 10)) # 1~10
7 print(datetime.now())
```

오늘은 '표준'만으로도 충분

- 표준 라이브러리 = 파이썬 설치에 기본 포함
- math: 수학 계산 / random: 난수 / datetime: 날짜·시간
- 실습에서 random으로 미니게임(숫자맞추기/로또)을 만들어볼 것
- LLM에게 '표준 라이브러리만 사용' 조건을 주면 과해지지 않는다

모듈과 import

```
1 # my_utils.py 파일이 있다고 가정
2 def to_f(c):
3     return c * 9/5 + 32
4
5 # main.py
6 import my_utils
7 print(my_utils.to_f(25))
8
9 # 또는
10 from my_utils import to_f
11 print(to_f(25))
```

핵심 포인트

- 모듈(module) = .py 파일 1개
- import는 '가져와서 쓰기'(중복 없이 재사용)
- from ... import ... 는 필요한 것만 가져오기
- 파일 이름과 모듈 이름이 연결된다 (my_utils.py → my_utils)

외부 라이브러리 설치(pip) 개요

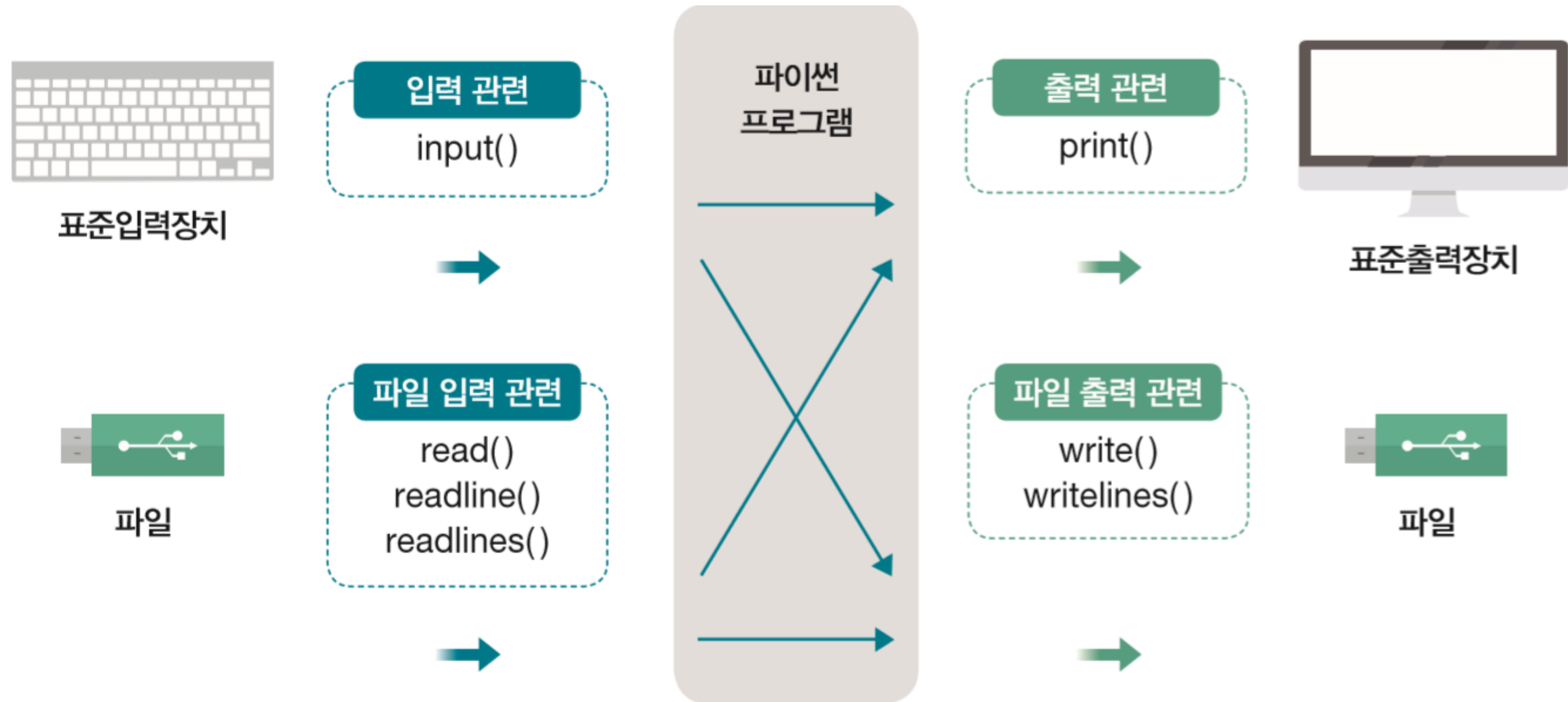
- 외부 라이브러리 = 파이썬에 기본 포함되지 않는 추가 기능
 - 설치: `pip install 패키지이름`
- 코랩에서 패키지 설치하는 경우
 - `!pip install 패키지이름`

라이브러리 선택 기준

- 문서/예제가 충분한가? (공식 문서 우선)
- 최근 업데이트가 있는가? (유지보수)
- 사용자/커뮤니티가 큰가? (검색/질문 가능)
- 라이선스/보안 이슈는 없는가?
- 수업에서는: '필요한 만큼만' 가져와 쓰기

파일 입출력의 개념

- 표준 입출력과 파일 입출력 함수



■ 파일 입출력의 기본 과정



■ 1단계 : 파일 열기

```
읽기용 : 변수명 = open("파일명", "r")  
쓰기용 : 변수명 = open("파일명", "w")
```

■ 2단계 : 파일 처리

■ 3단계 : 파일 닫기

- 1단계에서 open() 함수로 얻 변수명

```
변수명.close();
```

파일의 열기 모드

종류	설명
생략	r과 동일하다.
r	읽기 모드, 기본값이다.
w	쓰기 모드, 기존에 파일이 있으면 덮어쓴다.
r+	읽기/쓰기 겸용 모드이다.
a	쓰기 모드, 기존에 파일이 있으면 이어서 쓴다. append의 약어이다.
t	텍스트 모드, 텍스트 파일을 처리한다. 기본값이다.
b	이진 모드, 이진 파일을 처리한다.

■ 파일 입출력

- 경로(path) 기초

- ✓ 상대경로: 현재 폴더 기준(예: data/input.txt)
- ✓ 절대경로: 드라이브/루트부터 전체 경로
- ✓ 실습에서는 프로젝트 폴더 구조를 먼저 정한 후 경로 결정

- 파일 입출력 시 with 문 사용

- ✓ 파일 사용 후 자동으로 닫음
- ✓ 예외가 나도 자원 정리
- ✓ 파일 누수 방지

```
1 path = "data.txt"
2 with open(path, "r", encoding="utf-8") as f:
3     text = f.read()
4 print(text)
```

- 파일 입출력시 주의사항

- ✓ w는 기존 파일을 지움(주의)
- ✓ 문자 인코딩(utf-8) 지정 권장
- ✓ 예외 처리(FileNotFoundError)

■ 파일 입출력

● 파일 읽기

- ✓ read(): 전체를 한 번에
- ✓ readline(): 한 줄
- ✓ readlines(): 줄 리스트

● 파일 쓰기

- ✓ write는 문자열 1개
- ✓ writelines는 여러 줄
- ✓ (줄바꿈)을 직접 넣어야 함

● 예외처리 : FileNotFoundError

```
1 try:
2     with open("missing.txt", "r",
3               encoding="utf-8") as f:
4         print(f.read())
5 except FileNotFoundError:
6     print("파일이 없습니다")
```

■ 파일 입출력

● CSV 파일 읽기/쓰기

- ✓ Comma-Separated Values
- ✓ 표 형태(행/열)를 텍스트로 저장
- ✓ **csv 모듈로 읽기**

```
1 import csv
2 with open("data.csv", "r", encoding="utf-8") as f:
3     reader = csv.reader(f)
4     for row in reader:
5         print(row)
```

✓ **csv 모듈로 쓰기**

```
1 import csv
2 rows = [{"name", "score"}, {"kim", 90}]
3 with open("out.csv", "w", newline="",
4           encoding="utf-8") as f:
5     w = csv.writer(f)
6     w.writerows(rows)
```

04. 연료 소모량 계산 함수 [함수 활용]

문제: 선박의 속도(knot)를 입력하면 예상 연료 소모량을 계산해주는 함수를 만드세요.

•공식: 연료 소모량은 속도의 세제곱에 비례한다고 가정합니다 ($\text{Consumption} = k * \text{Speed}^3$). 여기서는 상수 $k = 0.05$ 를 사용합니다.

05. 선박 기자재 점검 로그 저장 (파일 입출력)

문제: 선박의 엔진 상태를 점검하고 그 결과를 inspection.txt 파일에 저장하는 프로그램을 작성하세요. 사용자로부터 점검 항목과 상태(정상/비정상)를 입력받아 기록합니다.

06. 선박 블록 무게 합계 계산기 ✦ 핵심 개념: 변수, 리스트, 반복문, 합계

선박은 수백 개의 '블록'으로 나뉘어 제작됩니다. 각 블록의 무게(톤) 데이터를 리스트로 만들고, 전체 선박의 총 무게와 평균 블록 무게를 계산하여 출력하세요.

07. 공정 진행률 계산 및 출력

✦ 핵심 개념: 딕셔너리, f-string, 조건문

선박 건조의 주요 공정(절단, 가공, 조립, 탑재, 의장, 도장)별 계획 물량과 실적 물량을 딕셔너리로 저장하고, 각 공정의 진행률(%)을 계산하여 출력하세요. 진행률이 100% 미만이면 '지연' 표시.

AI 빅데이터



데이터 분석

데이터 분석

- 의사결정: 숫자/기록을 근거로 판단하기
- 현업 데이터 예시: 설문, 매출, 센서, 로그, 출석/과제 기록
- 리스트/반복문만으로는 '표 형태 데이터' 처리에 한계가 있다
- NumPy/Pandas는 표준적인 데이터 처리 도구(속도+표현력)
- 데이터 분석의 시작
 - ✓ 분석 대상이 되는 데이터(자료)의 종류를 확인해야 한다.
 - ✓ 데이터의 종류에 따라 적용할 수 있는 분석 방법이 달라진다

선호 계절
여름
봄
가을
겨울
봄
가을

(a) 1차원데이터

이름	성별	성적
김철수	M	95
최서영	F	84
최예림	F	76
하재경	M	90
정다훈	M	85
민유진	F	90

(b) 2차원 데이터

그림 2-2 데이터의 예

표 2-2 파이썬 환경의 자료구조

배열의 차원	파이썬	넘파이	판다스
1차원 데이터	list, tuple, dictionary	ndarray(1차원)	Series
2차원 데이터	list, tuple	ndarray(2차원)	DataFrame

데이터의 예 (출처: AI 시대의 파이썬 데이터 분석)

Pandas 모듈

- 판다스(Pandas)는 파이썬에서 데이터 조작 작업을 단순하고 효율적으로 수행할 수 있도록 지원하는 강력한 라이브러리(패키지)이다.
- 소프트웨어 개발자이자 사업가인 웨스 맥키니(Wes McKinney)가 2008년에 개발하였다.
- "Pandas"는 "Panel Data"와 "Python Data Analysis"의 합성어이다.
- 판다스는 넘파이(NumPy) 라이브러리를 기반으로 구축되었으며, 1차원 배열 및 표 형태의 2차원 배열 데이터를 다룰 수 있도록 지원한다.
- 다양한 기능과 높은 사용자 편의성 덕분에, 파이썬 환경에서 데이터를 다루는 데이터 분석가, 과학자, 엔지니어에게 필수적인 도구로 인식된다.

Pandas: Series vs DataFrame

●시리즈(Series)

- ✓ 1차원 배열로서 단일 변수 데이터를 저장하는 데 사용된다.

●데이터프레임(DataFrame)

- ✓ 2차원 배열로서 다중 변수 데이터를 저장하는 데 사용된다.
- ✓ 데이터프레임은 여러 개의 1차원 배열(시리즈)을 세로 방향으로 묶어 놓은 형태를 가진다.

Series (1차원)

- 값 + index(라벨)
- 한 열(column)처럼 생각
- 예: 학생 점수 목록

DataFrame (2차원)

- 행(row) + 열(column) 표 구조
- 엑셀/CSV와 가장 가까움
- 데이터 분석의 중심 객체

Series 객체 생성

- 리스트를 직접 입력하여 생성하기

```
# Series 생성 (1차원 데이터)  
temperatures = pd.Series([23.5, 25.1, 19.8, 30.2])  
print(temperatures)
```

- 이미 생성된 리스트 객체를 이용하여 시리즈 생성하기

```
# Series 생성 (1차원 데이터)  
data = ['spring', 'summer', 'fall', 'winter']  
season = pd.Series(data)  
print(season)
```

Data Frame 객체 생성

- 리스트를 사용하여 데이터프레임 객체 생성하기

```
# DataFrame 생성 (2차원 데이터)
weather_data = pd.DataFrame([[23.5, 60, 0],
                             [25.1, 65, 0],
                             [19.8, 70, 5.2],
                             [30.2, 55, 0]])

print(weather_data)
```

- 딕셔너리를 사용하여 데이터프레임 객체 생성하기

```
# DataFrame 생성 (2차원 데이터)
weather_data = pd.DataFrame({
    '온도': [23.5, 25.1, 19.8, 30.2],
    '습도': [60, 65, 70, 55],
    '강수량': [0, 0, 5.2, 0]
}, index=['월', '화', '수', '목'])

print(weather_data)
```

Data Frame 객체 생성

- 외부 파일에서 불러와서 데이터프레임 객체 생성하기

```
df=pd.read_csv('data.csv', encoding='utf-8-sig')  
Print(df)
```

- 특정 조건을 필터링하여 불러오기

```
df_filtered = pd.read_csv(  
    file_name,  
    encoding='utf-8',  
    usecols=['name', 'korean'], # 특정 컬럼만  
    nrows=3 # 상위 3행만  
)  
print("=== 조건을 지정하여 불러오기 ===")  
print(df_filtered)
```

Data Frame 빠르게 살펴보기

- 데이터프레임 메소드를 사용하여 정보 확인하기
 - 1) head()로 대충 생김새 확인 -> 상위 5개 출력
 - 2) columns로 열 이름 확인
 - 3) info()로 타입/결측치 확인
 - 4) describe()로 통계 확인

AI 활용

1. 자재 데이터 관리 프로그램 만들기

- 자재 목록 엑셀 파일 생성 (샘플 데이터 생성 코드 요청)
- 자재 목록 엑셀 파일을 pandas로 읽어서 기본 정보 확인(컬럼 목록, 행 수, 상위 5개 행, 기본 통계를 출력)
- 자재 데이터에서 '단가 × 재고수량'으로 재고 금액 컬럼을 추가하고, 금액 기준으로 내림차순 정렬하세요. 재고 금액이 1,000만 원 이상인 자재만 필터링하여 출력

강의 자료 : <https://github.com/misun0314/AI-Python/>

질문 : misun@mnu.ac.kr